

Understanding the OWASP Top 10

A Practical Study of Five Critical Web Application Security Risks

Muhammad Zohaib Shabbir

Vortex Tech Cyber Security Internship — Week 1 of 4 — July 2026

Contents

0.1	1. Executive Summary	1
0.2	2. Introduction to the OWASP Top 10	2
0.3	3. Research Methodology	3
0.4	4. A01:2025 — Broken Access Control	3
0.5	5. A02:2025 — Security Misconfiguration	5
0.6	6. A04:2025 — Cryptographic Failures	7
0.7	7. A05:2025 — Injection	8
0.8	8. Cross-Site Scripting (XSS) — A Special Case of Injection	10
0.9	9. Comparative Summary Table	11
0.10	10. Personal Reflection	12
0.11	11. Conclusion	13
0.12	12. References	13

0.1 1. Executive Summary

This report was produced as the Week 1 deliverable for the Vortex Tech Cyber Security Internship. Its purpose is to build a strong theoretical foundation in web application security by researching five vulnerability categories from the OWASP (Open Web Application Security Project) Top 10, the industry-standard awareness document that security professionals, auditors, and developers use to prioritize application security risks.

Rather than simply defining each vulnerability, this report explains *what* each risk is in plain language, *how* an attacker would realistically exploit it, walks through a *real-world breach* that resulted from it, and lays out *practical, developer-facing mitigations*. Five categories were selected: Broken Access Control, Security Misconfiguration, Cryptographic Failures, Injection, and Cross-Site Scripting (XSS) — the last of which is now formally treated as a specialized sub-pattern of Injection but is significant enough, historically and practically, to merit its own detailed case study.

A notable finding during research is that OWASP released a new edition of the Top 10 — the **OWASP Top 10:2025** — which became the current official list after its final release in early 2026. This is more recent than most existing beginner guides and training material, which still reference the 2021 edition. This report uses the 2025

categories and rankings throughout, while also mapping them back to their 2021 equivalents so the material stays useful regardless of which edition a reader is more familiar with.

0.2 2. Introduction to the OWASP Top 10

OWASP is a nonprofit, community-driven foundation whose mission is to improve the security of software. Its best-known project, the OWASP Top 10, is a standard awareness document that represents broad industry consensus on the most critical security risks facing web applications. It is not a formal vulnerability checklist, a certification, or a legal compliance standard — it is a data-driven ranking, built primarily from two inputs:

- **Contributed testing data** from security vendors and organizations, covering hundreds of thousands of tested applications and hundreds of Common Weakness Enumerations (CWEs).
- **A practitioner survey**, which allows the security community to flag emerging risks that may not yet be well represented in historical testing data (this is how newer risks, like Server-Side Request Forgery in 2021 and Software Supply Chain Failures in 2025, first entered the list).

The list has been revised roughly every three to four years since it was first published in 2003: 2004, 2007, 2010, 2013, 2017, 2021, and most recently **2025**. Each revision reflects how real attacks and application architectures have shifted — for example, the rise of cloud infrastructure, APIs, microservices, and CI/CD pipelines has steadily changed which weaknesses are most exploited in practice.

0.2.1 2.1 What changed between the 2021 and 2025 editions

Because this internship task was originally scoped against the 2021 list, it is worth explicitly noting what changed and why this report follows the newer edition instead:

- **Broken Access Control** stayed at #1, and now explicitly absorbs **Server-Side Request Forgery (SSRF)**, which was its own standalone category (A10) in 2021.
- **Security Misconfiguration** jumped from #5 (2021) to **#2** (2025) — continuous deployment without continuous configuration checking is now recognized as a major, widespread exposure window.
- **Software Supply Chain Failures** is a new category at #3, expanding on the old “Vulnerable and Outdated Components” to cover build systems, CI/CD pipelines, and third-party dependency risk more broadly.
- **Cryptographic Failures** dropped from #2 to **#4**.
- **Injection** (which now includes Cross-Site Scripting) dropped from #3 to **#5** — not because injection stopped mattering, but because parameterized queries and modern frameworks have measurably reduced how often it is found, even though it remains extremely high-volume in raw CVE counts.
- **Insecure Design** moved from #4 to #6.
- **Identification and Authentication Failures** was renamed **Authentication Failures** and stayed around #7.

- **Software and Data Integrity Failures** stayed at #8, but is now distinguished from Software Supply Chain Failures (A08 focuses on trust and integrity verification of code/data at the artifact level; A03 focuses on the broader dependency and build ecosystem).
- **Security Logging and Monitoring Failures** was renamed **Security Logging and Alerting Failures** and stayed at #9, with more emphasis on whether alerts actually fire, not just whether logs exist.
- **Mishandling of Exceptional Conditions** is a brand-new category at #10, covering uncaught exceptions, race conditions, and missing error-handling policy — it replaces SSRF’s old A10 slot, since SSRF was folded into Broken Access Control.

The table in Section 9 lays this mapping out side-by-side for quick reference.

0.3 3. Research Methodology

This report follows the step-by-step approach outlined in the Week 1 task brief:

1. Consulted the official OWASP Top 10 documentation (owasp.org/Top10) for both the 2021 and 2025 editions to confirm the current, authoritative category list and ranking data.
2. Cross-referenced multiple independent security vendor analyses (GitLab, Orca Security, Qualys, SentinelOne, Fastly, Indusface, and others) to understand *why* categories moved and what practical changes they imply for developers.
3. For each selected vulnerability, researched a well-documented, publicly reported real-world breach, using primary and reputable secondary sources (breach disclosures, Krebs on Security, Wikipedia summaries of public record, incident post-mortems, and vendor case studies).
4. Verified prevention guidance against OWASP’s own published recommendations and cross-checked it against independent practitioner sources to avoid relying on a single opinion.
5. Wrote all explanations in original language, translating technical OWASP wording into plain, beginner-friendly explanations, per the brief’s instruction to write “as if explaining it to a friend with no security background.”

Estimated time spent on research, writing, and formatting: approximately 7 hours, in line with the brief’s 6–8 hour estimate.

0.4 4. A01:2025 — Broken Access Control

OWASP rank: #1 (also #1 in 2021) · **Also covers (2025):** Server-Side Request Forgery (SSRF), Broken Object Level Authorization (BOLA), Broken Function Level Authorization (BFLA)

0.4.1 4.1 What it is

Access control is the set of rules that decide what a logged-in user is *allowed* to do and see. Broken Access Control happens when an application checks *whether* you're logged in, but fails to properly check *what you're allowed to access* once you are. In plain terms: the front door has a lock, but once you're inside the building, you can walk into any room — including ones that aren't yours — because nobody is checking which rooms you're actually allowed in.

This is currently the single most common serious flaw found in tested web applications, and in the 2025 data it affects roughly one in every twenty-five applications tested.

0.4.2 4.2 How an attacker exploits it

The most common real-world pattern is an **Insecure Direct Object Reference (IDOR)**: a URL or API request contains an identifier — like `?invoice_id=1005` — and the server hands back whatever record matches that ID without checking whether the requesting user actually owns it. An attacker (or even a curious ordinary user) simply changes the number to 1006, 1007, and so on, and reads other people's data with no hacking tools beyond a web browser.

A related pattern, **Broken Function Level Authorization**, happens when an application hides an admin feature from the menu for regular users but doesn't actually block the underlying request — so a regular user who guesses or discovers the admin URL (e.g., `/admin/deleteUser`) can call it directly and it just works, because the server never re-checks the user's role.

Because 2025 folded SSRF into this category, it's also worth noting that pattern: an attacker supplies a URL to a server-side feature (like a "fetch this image from a link" tool) that instead points the server at an internal-only address, tricking the server into making a request on the attacker's behalf to a resource it should never have been able to reach directly (such as a cloud provider's internal metadata service).

0.4.3 4.3 Real-world example: First American Financial Corp. (2019)

In May 2019, a real estate developer discovered that First American Financial Corp., a major U.S. title insurance company, had exposed approximately **885 million records** — mortgage documents, bank account numbers, Social Security numbers, wire transfer receipts, and driver's license images — going back to 2003. The breach happened because a website design flaw let anyone view sensitive documents by simply changing a number in a web link. The underlying documents were referenced through sequential, easily guessable identifiers, so once someone had one valid document link, incrementing the number retrieved a completely different customer's private file — with no login required.

A Barracuda Networks security director described this exact flaw as an extremely common programming mistake, noting that the resulting trove of sensitive information could fuel identity theft, spear phishing, or business email compromise. First American was later fined by the New York State Department of Financial Services —

notably the first enforcement action brought under that state’s newly introduced cybersecurity regulations at the time. No password had to be cracked and no malware was involved; the entire breach was possible because the application never verified that the person requesting a document was actually authorized to see it.

0.4.4 4.4 Prevention and mitigation

- **Deny by default.** Every request to a protected resource should require an explicit, server-side check confirming the requester owns or is authorized to view that specific record — never assume authorization just because a request “looks” legitimate.
 - **Use unpredictable, non-sequential identifiers** (such as UUIDs) for sensitive records, so guessing a valid ID for another user’s data becomes computationally infeasible even as a secondary layer of defense.
 - **Implement a centralized authorization check** (a single reusable function or middleware layer) rather than scattering access checks across many individual routes, where it is easy to forget one.
 - **Log and alert on access control failures**, such as repeated 403/404 responses from the same user, which often indicate someone is probing for accessible object IDs.
 - For SSRF specifically: **allowlist outbound destinations** the server is permitted to call, and block requests to internal/metadata IP ranges (e.g., 169.254.169.254) by default.
-

0.5 5. A02:2025 — Security Misconfiguration

OWASP rank: #2 (up from #5 in 2021) · **Related CWEs include:** default credentials, verbose error messages, unnecessary features enabled, missing security headers, outdated software

0.5.1 5.1 What it is

Security Misconfiguration is what happens when a system is technically secure *by design* but insecure *in practice* — because of how it was actually set up. This includes leaving default admin passwords unchanged, leaving debugging features or verbose error pages enabled in production, leaving cloud storage buckets or databases open to the public internet, or granting a service far more permission than it actually needs to do its job.

Think of it like buying a house with a great alarm system and then never turning it on, or leaving a spare key under the doormat “just in case.” The security feature exists — it’s just not correctly enabled or configured.

0.5.2 5.2 How an attacker exploits it

Attackers routinely run automated scanners across the internet looking for exactly this: exposed admin panels still using default logins, cloud storage buckets with pub-

lic read/write access, unpatched software with known vulnerabilities, or servers that reveal too much information in error messages (like a full stack trace showing internal file paths or database structure). None of this requires “hacking” in the traditional sense — it requires noticing what an organization forgot to lock down.

A more advanced but increasingly common pattern involves **cloud identity and access misconfiguration**: a supporting service is granted broader permissions than it needs, and if that service can be tricked into acting on an attacker’s behalf (see the SSRF pattern in Section 4), the attacker inherits all of that service’s excess permissions in one step.

0.5.3 5.3 Real-world example: Capital One (2019)

In one of the most instructive cloud-security breaches on record, a former Amazon Web Services engineer exploited a **misconfigured web application firewall (WAF)** running inside Capital One’s AWS cloud environment. The attacker sent a server-side request forgery command through the misconfigured firewall to reach AWS’s internal EC2 metadata service and retrieve temporary cloud credentials, which were then used to access and exfiltrate personal information belonging to roughly 106 million credit card applicants across the United States and Canada.

The core misconfiguration was twofold: the firewall itself could be tricked into querying AWS’s internal metadata endpoint, and the cloud role attached to that firewall had far more permission than it needed — it was able to list and read the contents of any storage bucket in the account. Investigators later concluded that the greatest failure was the absence of proper monitoring and alerting that could have flagged the unusual, large-scale data access as it was happening. Capital One paid an \$80 million civil penalty to federal regulators and settled a related class action for roughly \$190 million. In direct response, AWS introduced a hardened version of its metadata service (IMDSv2) that requires session-based authentication, specifically to make this class of attack much harder industry-wide.

0.5.4 5.4 Prevention and mitigation

- **Harden before deployment.** Disable unused features, sample applications, default accounts, and verbose debug/error output before anything reaches production.
- **Apply least privilege everywhere** — a supporting service (like a WAF, proxy, or automation script) should only ever hold the minimum cloud permissions it needs to function, never broad account-wide access.
- **Automate configuration auditing.** Use infrastructure-as-code scanning and continuous compliance tools so a misconfiguration introduced in one deployment is caught before the next, rather than discovered months later.
- **Segment and monitor cloud metadata services;** adopt session-authenticated metadata access (e.g., IMDSv2 on AWS) so a request-forwarding bug can’t silently leak credentials.
- **Patch and update on a defined cadence,** and keep a current software/component inventory so nothing is “forgotten” and left running an old, vulnerable version indefinitely.

0.6 6. A04:2025 — Cryptographic Failures

OWASP rank: #4 (down from #2 in 2021, previously named “Sensitive Data Exposure”) · **Related CWEs include:** use of weak/broken algorithms, hard-coded keys, missing encryption in transit, reversible password storage

0.6.1 6.1 What it is

Cryptographic Failures cover any situation where sensitive data — passwords, financial details, personal information, health records — isn’t properly protected by encryption, or is protected the *wrong* way. This is a “root cause” category: the failure is in how cryptography is used (or not used), and the resulting exposure of sensitive data is the *symptom*, which is why OWASP renamed this category away from its older, symptom-focused name of “Sensitive Data Exposure.”

A common and surprisingly persistent mistake is confusing **encryption** (which is reversible if you have the key) with **hashing** (which is designed to be one-way and irreversible) — using the wrong one for passwords specifically is one of the most damaging cryptographic mistakes an engineering team can make.

0.6.2 6.2 How an attacker exploits it

If an attacker gains access to a database (through any other vulnerability, or an insider), what they find determines how damaging the breach is. If passwords were properly hashed with a modern, “slow” algorithm designed for passwords (like bcrypt, scrypt, or Argon2) and individually salted, cracking them back into plaintext is extremely resource-intensive, often practically infeasible at scale. But if passwords were merely *encrypted* with a shared, reusable key — or worse, hashed with a fast, general-purpose algorithm like unsalted MD5 or SHA-1 — an attacker who obtains the key or runs the data through cracking tools and pre-computed lookup tables can recover large numbers of real passwords quickly.

Attackers also intercept data that is never encrypted in transit at all — for example, a login form submitted over plain HTTP instead of HTTPS can be read directly off the network by anyone positioned to observe the traffic, such as on shared public Wi-Fi.

0.6.3 6.3 Real-world example: Adobe (2013)

In October 2013, Adobe disclosed that attackers had accessed a backup authentication system containing data for what was ultimately confirmed to be **153 million user accounts**. Independent researchers determined that Adobe was not following industry best practice: rather than being hashed, the passwords had instead been encrypted using a decades-old symmetric cipher (Triple DES).

The specific implementation mistake made things dramatically worse. A security researcher analyzing the leaked data noted that because Adobe used symmetric encryption instead of hashing, applied it in a mode where identical inputs produce identical

outputs, and reused the same encryption key for every single password, combined with millions of users who had left plaintext password hints, this weakness — even without recovering the master encryption key — still allowed researchers to reconstruct a “top 100 passwords” list with high confidence. Nearly 1.9 million accounts, for example, turned out to be using “123456.” Because the encryption scheme leaked structural information (such as password length and whether two users shared the same password), it defeated much of the protection encryption is supposed to provide, even before anyone cracked the underlying key.

0.6.4 6.4 Prevention and mitigation

- **Never encrypt passwords — hash them**, using a modern, purpose-built, deliberately slow algorithm (bcrypt, scrypt, or Argon2) combined with a unique salt per user.
 - **Classify data first**. Identify which data is sensitive (financial, health, personal identifiers, credentials) according to applicable privacy laws or business risk, and apply encryption specifically where it matters, rather than treating it as an afterthought.
 - **Enforce encryption in transit everywhere** — HTTPS/TLS should be mandatory site-wide, not just on login or checkout pages, with modern TLS versions and forward secrecy.
 - **Never eliminate the reversibility problem with a single shared key**. Use proper key management: per-purpose keys, rotation policies, and secure key storage (such as a hardware security module or managed key vault), rather than embedding keys directly in source code.
 - **Avoid storing sensitive data you don’t actually need** — data that was never collected in the first place can never be breached.
-

0.7 7. A05:2025 — Injection

OWASP rank: #5 (down from #3 in 2021) · **Includes:** SQL injection, command injection, code injection (e.g., OGNL/expression-language injection), and — historically as a separate category — Cross-Site Scripting

0.7.1 7.1 What it is

Injection happens when an application takes input from a user and passes it directly into a command, query, or interpreter without properly separating “data” from “instructions.” The classic example is SQL injection: if a login form builds a database query by directly pasting in whatever the user typed, a user can type something that isn’t a name or password at all, but a fragment of database command language — and the database, unable to tell the difference, executes it as though the application itself had written it.

0.7.2 7.2 How an attacker exploits it

A textbook SQL injection example: imagine a login form that builds a query like: `SELECT * FROM users WHERE username = 'INPUT' AND password = 'INPUT'`

If the application inserts user input directly without safeguards, an attacker can type `' OR '1'='1` as the username. The resulting query becomes logically always-true, and the database returns the first user record in the table — often letting the attacker log in without ever knowing a valid password. More advanced injection attacks can extract entire tables of data, modify records, or in the worst cases, execute operating-system-level commands on the underlying server.

Not all injection is SQL, either. **Code injection** occurs when an application passes user-controlled input into something that gets interpreted and *executed* as code rather than as inert data — for example, an expression language parser that’s supposed to only process trusted internal templates, but is accidentally exposed to attacker-controlled input via an HTTP header.

0.7.3 7.3 Real-world example: Equifax (2017)

The Equifax breach is one of the most consequential injection-related incidents on record, exposing the personal data of roughly 147.9 million Americans, along with millions of UK and Canadian residents — including Social Security numbers, birth dates, addresses, and in some cases driver’s license and credit card numbers.

The breach began when attackers exploited CVE-2017-5638, a critical flaw in the open-source Apache Struts web framework that Equifax’s online dispute portal relied on. The flaw allowed remote code execution through a maliciously crafted HTTP Content-Type header. Technically, this was a **code injection** vulnerability: Struts’ Jakarta Multipart parser failed to treat an invalid Content-Type header as plain text, and instead processed it as OGNL — an expression language — meaning attacker-supplied text was executed as code on the server.

What turned a known, patchable bug into a catastrophic breach was Equifax’s own process failure: Apache had disclosed the vulnerability and released a patch on March 7, 2017; the Department of Homeland Security notified Equifax, Experian, and TransUnion of the issue the very next day; and an internal email went out to Equifax administrators directing them to apply the patch — but Equifax’s own security scans on March 15, 2017 failed to detect that the vulnerable systems were still exposed. The vulnerability remained unpatched until July 29, 2017, when Equifax’s security team finally noticed suspicious network traffic on the dispute portal and acted. By then, attackers had been inside the network for roughly two and a half months, using the initial code injection foothold to steal internal credentials and move laterally into the core databases.

0.7.4 7.4 Prevention and mitigation

- **Use parameterized queries or prepared statements for all database access** — never build SQL queries by directly concatenating user input into a string. This single change eliminates the vast majority of classic SQL injection risk.

- **Treat all user input as untrusted data, never as code or commands**, and validate it against a strict allowlist of expected formats wherever possible.
 - **Keep frameworks and dependencies patched on a defined, enforced schedule**, and maintain an accurate inventory of what open-source components are in use — the Equifax breach was ultimately a patching-process failure as much as a technical one.
 - **Apply the principle of least functionality**: don't expose powerful features (like expression-language evaluation) to any input path that could realistically be reached by an untrusted user.
 - **Use automated dependency and vulnerability scanning** in the build pipeline so known-vulnerable components are flagged before deployment, not discovered after a breach.
-

0.8 8. Cross-Site Scripting (XSS) — A Special Case of Injection

OWASP status: Standalone category (A03) through the 2017 edition; consolidated into the Injection category (A05) in both 2021 and 2025 · **Still one of the highest-volume vulnerability types found in real-world testing**

0.8.1 8.1 What it is

Cross-Site Scripting is, structurally, an injection flaw — but instead of injecting commands into a database or server, the attacker injects malicious script into a web page that *other users* will later view. When that page loads in a victim's browser, the malicious script runs with the same trust and access as the legitimate site — meaning it can read cookies, steal session tokens, or silently perform actions on the victim's behalf.

0.8.2 8.2 How an attacker exploits it

The most dangerous variant is **stored XSS**: the malicious script is saved somewhere on the server — a comment field, a user profile bio, a forum post — and served back to *every* visitor who views that content, with no further action needed by the attacker. **Reflected XSS**, by contrast, requires tricking a specific victim into clicking a crafted link where the malicious script is embedded directly in the URL and echoed back by the vulnerable page.

0.8.3 8.3 Real-world example: The Samy Worm, MySpace (2005)

In October 2005, a 19-year-old MySpace user named Samy Kamkar discovered a **stored XSS vulnerability** in MySpace's profile customization feature. MySpace allowed users to style their profiles with custom HTML and CSS, and while it filtered out certain obviously dangerous tags and keywords, its filtering was incomplete, and Samy found multiple ways to bypass it. By breaking the word "javascript" across two lines inside a CSS property, he found a way to get a browser to still interpret and execute it as executable code despite MySpace's filter blocking the literal word.

He then combined this XSS bug with AJAX requests that, when triggered simply by *viewing* his profile, silently sent a friend request to Samy and added the phrase “but most of all, samy is my hero” to the visitor’s own profile — and, critically, copied the entire malicious script into the visitor’s profile too, making the exploit self-replicating. Because every new infected profile spread the payload to everyone who viewed it, growth was exponential: in under 20 hours, over one million MySpace users had unknowingly run the payload, making it the fastest-spreading piece of malicious code of its kind at that time. MySpace was ultimately forced to take the entire site offline to contain the outbreak and clean the affected profiles.

Although the Samy Worm was intentionally harmless — its entire “payload” was a friend request and a flattering sentence — the researcher who built it noted that a more malicious actor could have used the exact same technique to take complete control of every affected account, since a stored XSS payload can run any script the attacker chooses, not just a benign one. The incident is widely credited with pushing XSS from a theoretical concern into a mainstream, urgently-treated security priority across the industry.

0.8.4 8.4 Prevention and mitigation

- **Encode output based on context.** Data inserted into HTML, JavaScript, or a URL each require different, context-aware escaping — a single generic filter is not sufficient.
- **Never rely on blacklisting specific tags or keywords.** MySpace’s failure was a textbook example: blocking the literal word “javascript” did nothing against a payload split across two lines. Use an allowlist-based HTML sanitizer instead, or avoid allowing raw HTML input entirely.
- **Set a strict Content Security Policy (CSP).** A properly configured CSP can prevent injected scripts from executing at all, even if a filter bypass is later found — this is considered the strongest modern defense against XSS.
- **Mark cookies `HttpOnly` and `Secure`** so that even if a script does execute, it cannot read session cookies directly, limiting the damage of an XSS bug that does slip through.
- **Use modern frameworks that auto-escape by default** (such as React, Vue, or Angular’s default template rendering), which close off most accidental XSS simply by how they render user-supplied content.

0.9 9. Comparative Summary Table

Vulnerability	2025 Rank	2021 Rank	Case Study	Root Cause	Primary Fix
Broken Access Control	A01 (#1)	A01 (#1)	First American Financial, 2019 (~885M records)	No server-side ownership check on document IDs	Deny by default; centralized authorization checks
Security Misconfiguration	A02 (#2)	A05 (#5)	Capital One, 2019 (~106M records)	Misconfigured WAF + over-permissioned cloud role	Least privilege; automated config auditing
Cryptographic Failures	A04 (#4)	A02 (#2)	Adobe, 2013 (~153M accounts)	Passwords encrypted, not hashed; single reused key	Hash with bcrypt/Argon2 + per-user salt
Injection	A05 (#5)	A03 (#3)	Equifax, 2017 (~147.9M individuals)	Unpatched Apache Struts; OGNL code injection	Parameterized queries; enforced patch cadence
Cross-Site Scripting	(<i>within A05</i>)	(<i>within A03</i>)	Samy Worm / MySpace, 2005 (1M+ profiles)	Incomplete blacklist filtering of script content	Output encoding; CSP; allowlist sanitization

0.10 10. Personal Reflection

Researching this report shifted how I think about the OWASP Top 10 — from a list to memorize into a genuine pattern I can now recognize across very different incidents. The vulnerability that surprised me most was **Security Misconfiguration**, specifically the Capital One breach. I expected a breach of that scale to require some kind of sophisticated exploit or zero-day vulnerability, but the actual root cause was closer to an administrative oversight: a firewall was set up with more trust and more permission than it ever needed, and a single request-forwarding bug was enough to cash in on all of it. It reframed misconfiguration, in my mind, from a “minor” issue into something that can be just as devastating as a classic exploit — arguably more dangerous, because it doesn’t require the attacker to be especially skilled, only observant.

I was also struck by how often these breaches trace back to a **process failure rather than a purely technical one**. Equifax had a patch available two months before attackers used the exact flaw it fixed. Capital One’s misconfiguration reportedly existed in production for an extended period before anyone caught it. This told me that

secure coding practices only work if they're paired with equally disciplined operational habits — patch management, permission audits, and monitoring that someone actually reviews. Learning the OWASP Top 10 in this depth has made it clear that application security isn't a single skill, but a discipline that spans writing correct code, configuring infrastructure carefully, and maintaining both over time. It has motivated me to go deeper into hands-on, practical security testing in the coming weeks of this internship, rather than stopping at theory.

0.11 11. Conclusion

The five vulnerabilities examined in this report — Broken Access Control, Security Misconfiguration, Cryptographic Failures, Injection, and Cross-Site Scripting — remain, in one form or another, at or near the top of the OWASP Top 10 across every edition of the list, including the newly released 2025 version. Despite years of tooling improvements, secure frameworks, and industry awareness, each of the case studies in this report — First American Financial, Capital One, Adobe, Equifax, and the Samy Worm — shows that these are not abstract, theoretical risks. They are recurring, well-understood failure patterns that continue to cause some of the largest data breaches in history, usually because a known, preventable safeguard was missing, disabled, or simply never checked. Understanding these patterns, and the concrete defensive habits that prevent them, is foundational to any career in application security or software engineering.

0.12 12. References

- OWASP Foundation. (2025). *OWASP Top 10:2025*. <https://owasp.org/Top10/2025/>
- OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/2021/>
- OWASP Foundation. *OWASP Top Ten Web Application Security Risks*. <https://owasp.org/www-project-top-ten/>
- Apache Software Foundation. (2017). *Statement on the Equifax Data Breach*. <https://news.apache.org/foundation/entry/media-alert-the-apache-software>
- Equifax Inc. (2017). *Equifax Releases Details on Cybersecurity Incident, Announces Personnel Changes*. <https://investor.equifax.com/news-events/press-releases/detail/237/>
- Wikipedia contributors. *2017 Equifax data breach*. [https://en.wikipedia.org/wiki/2017_Equifax](https://en.wikipedia.org/wiki/2017_Equifax_data_breach)
- Krebs, B. (2019). *What We Can Learn from the Capital One Hack*. Krebs on Security. <https://krebsonsecurity.com/2019/08/what-we-can-learn-from-the-capital-one-hack/>
- APIsec. (2021). *What First American's \$885M Leak Teaches API Teams*. <https://www.apisec.ai/blog/first-american-financial-885m-data-vulnerability>
- SecurityWeek. (2019). *First American Financial Exposed Millions of Sensitive Documents*. <https://www.securityweek.com/first-american-financial-exposed-millions-sensitive-documents/>
- CSO Online. (2013). *Adobe confirms stolen passwords were encrypted, not hashed*. <https://www.csoonline.com/article/540070/>

- InfoWorld. (2013). *Adobe confirms millions of stolen passwords were not secured properly*. <https://www.infoworld.com/article/2612571/>
- Wikipedia contributors. *Samy (computer worm)*. [https://en.wikipedia.org/wiki/Samy_\(computer_worm\)](https://en.wikipedia.org/wiki/Samy_(computer_worm))
- Vice / Motherboard. (2016). *The MySpace Worm that Changed the Internet Forever*. <https://www.vice.com/en/article/the-myspace-worm-that-changed-the-internet-forever/>
- BetaNews. (2005). *Cross-Site Scripting Worm Hits MySpace*. <https://betanews.com/2005/10/cross-site-scripting-worm-hits-myspace/>
- GitLab. (2026). *OWASP Top 10 2025: What's Changed and Why It Matters*. <https://about.gitlab.com/blog/2025-owasp-top-10-whats-changed-and-why-it-matters/>
- Orca Security. (2025). *OWASP Top 10 2025: Key Changes and What They Mean for Application Security*. <https://orca.security/resources/blog/owasp-top-10-2025-key-changes/>

Note: For the purposes of this internship assignment, all breach summaries above are original paraphrases and analyses written from publicly reported information; readers seeking primary-source detail should consult the linked references directly.